

[\[Return to Main Page\]](#)

Division (32-bit) by Garth Wilson

[\[Up to Source Code Repository\]](#)

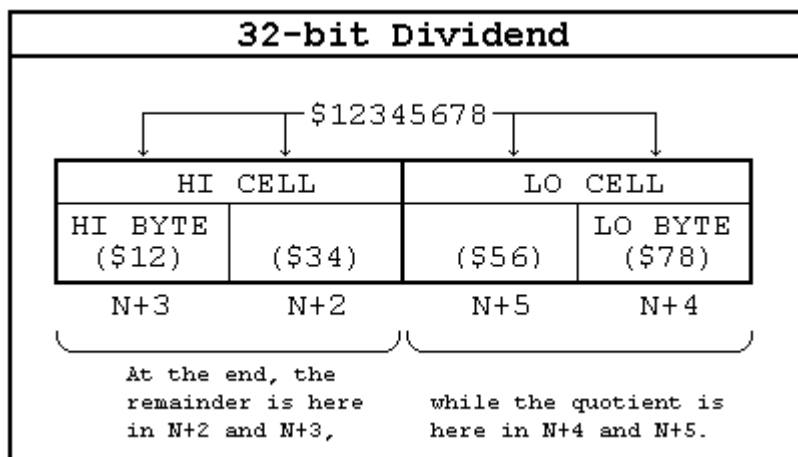
How to divide a 32-bit dividend by a 16-bit divisor.

By Garth Wilson (wilsonmines@dslexreme.com), 9 Sep 2002.

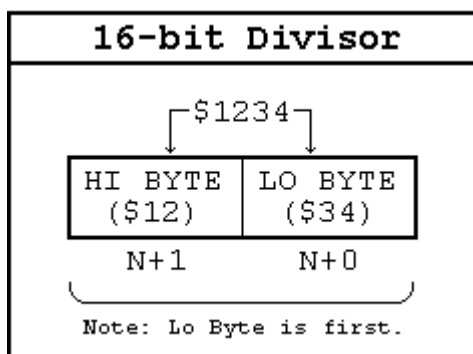
This shows an unsigned division of a 32-bit dividend by a 16-bit divisor, where you end up with a 16-bit quotient and a 16-bit remainder.

The particular reason for showing it is that there's a bug in the way it's often done in Forth (UM/MOD) and possibly other languages too. The faulty common Forth version is shown after this listing, with Forth ZP data stack indexing. The Forth-specific header structure is not shown in any of these.

To start, the 32-bit dividend is in N+2 (byte 2), N+3 (byte 1, or hi byte), N+4 (byte 4, or lo byte), & N+5 (byte 3).



The 16-bit divisor is in N+0 and N+1 (lo byte first, as is usual for the 6502). "N" is the beginning address of a small scratchpad area of RAM.



At the end, the 16-bit quotient is in N+4 & N+5, and 16-bit remainder is in N+2 & N+3 both with low byte first, as usual for 6502, like the divisor shown above. In other words, the remainder and quotient end up in the same memory bytes that originally held the dividend.

- N+6 is used for an intermediate scratchpad byte.
- X is used for loop-counting.
- Y is used for temporary byte storage.
- Overflow and divide by zero are shown by FFFF in both quotient and remainder.

- Again, remember input and output are unsigned. UM/MOD in Forth (pronounced "you-em-slash-mod") means Unsigned, Mixed-precision (double precision dividend and single-precision divisor and results) division and finding the mod (remainder). It's a building block for most parts of Forth that do any division. It's not necessary to have any interest in Forth to benefit from this however.

```
; ----- ZP RAM Variables -----
.PAGE0

N:          BLKB  7      ; 7 bytes of ZP for input, output, and scratchpad
CARRY:      BLKB  1      ; 1 byte  of ZP for 17th bit (Could be called N+7)

COMMENT
N & N+1 could be called "DIVISOR" (and DIVISOR+1).  Since the answer ends up
in the same bytes that originally held the 32-bit dividend, it becomes a
little harder to come up with good variable names for other bytes of N unless
you have more than one name for the same locations and remember that they are
indeed the same locations.

This was originally for my 65C02 Forth kernel, where the input and output is
through the data stack, which is in ZP and indexed by X.  This shows how you'd
do the actual division if you moved it to the Forth ZP scratchpad area for
primitives (ie, code definitions).  This area is often simply called N.  In
Forth, we don't normally move all this to and from this area for division,
because the time needed to move it is greater than the time saved by not
having to index by X; but the division process itself takes a little less
explaining this way, and you don't need to know anything about Forth to use
this routine.  Refer to the diagrams above for what is in N to N+6.
END_COMMENT

; ----- CODE -----

START:  SEC          ; Detect overflow or /0 condition.
        LDA      N+2  ; Divisor must be more than high cell of dividend. To
        SBC      N    ; find out, subtract divisor from high cell of dividend;
        LDA      N+3  ; if carry flag is still set at the end, the divisor was
        SBC      N+1  ; not big enough to avoid overflow. This also takes care
        BCS      oflo$ ; of any /0 condition. Branch if overflow or /0 error.
                        ; We will loop 16 times; but since we shift the dividend
        LDX      #11H ; over at the same time as shifting the answer in, the
                        ; operation must start AND finish with a shift of the
loop:   ROL      N+4  ; Move low cell of dividend left one bit, also shifting
        ROL      N+5  ; answer in. The 1st rotation brings in a 0, which later
                        ; gets pushed off the other end in the last rotation.
        DEX
        BEQ      end$ ; Branch to the end if finished.

        ROL      N+2  ; Shift high cell of dividend left one bit, also
        ROL      N+3  ; shifting next bit in from high bit of low cell.
        STZ      CARRY ; Zero old bits of CARRY so subtraction works right.
        ROL      CARRY ; Store old high bit of dividend in CARRY. (For STZ
                        ; one line up, NMOS 6502 will need LDA #0, STA CARRY.)
        SEC      ; See if divisor will fit into high 17 bits of dividend
        LDA      N+2  ; by subtracting and then looking at carry flag.
        SBC      N    ; First do low byte.
        STA      N+6  ; Save difference low byte until we know if we need it.
        LDA      N+3  ;
        SBC      N+1  ; Then do high byte.
        TAY      ; Save difference high byte until we know if we need it.
        LDA      CARRY ; Bit 0 of CARRY serves as 17th bit.
        SBC      #0   ; Complete the subtraction by doing the 17th bit before
        BCC      loop ; determining if the divisor fit into the high 17 bits
```

```

; of the dividend.  If so, the carry flag remains set.
LDA    N+6      ; If divisor fit into dividend high 17 bits, update
STA    N+2      ; dividend high cell to what it would be after
STY    N+3      ; subtraction.
BRA    loop     ; Always branch.  NMOS 6502 could use BCS here.

oflo$: LDA    #FFH    ; If overflow occurred, put FF
STA    N+2      ; in remainder low byte
STA    N+3      ; and high byte,
STA    N+4      ; and in quotient low byte
STA    N+5      ; and high byte.
end$:  RTS
;-----

```

This is the end of the corrected version. Now we'll look at the common version that gives the wrong results.

Common UM/MOD gives incorrect results with high numbers. Here are some examples I ran after figuring out where the problem lay in a square root word I wrote:

Dividend	Divisor	Quotient	Remainder (Wrong quot & rem)
70000000	FFFF	0	0
60000000	FFFF	0	0
20000000	FFFF	0	0
20000000	EFFF	0	0
7FFFFFFF	EFFF	8000	7FFF
7FFFFFFF	FFFF	8000	7FFF
7FFFFFFF	8FFF	E001	5000
7FFFFFFF	800F	FFE2	1C1
90000000	A000	0	0

Actual division part of UM/MOD in many 6502 Forths is as follows. It's slightly faster than the fix detailed above, but what good is that if it doesn't always give the right answers?

To start, ZP data stack usage makes:

0,X & 1,X = 16-bit divisor

2,X & 3,X = hi 16 bits of dividend

4,X & 5,X = lo 16 bits of dividend

N holds 8-bit loop counter

At the end:

2,X & 3,X = 16-bit quotient

4,X & 5,X = 16-bit remainder

```

START:  LDA    4,X
        LDY    2,X
        STY    4,X
        ASL    A
        STA    2,X
        LDA    5,X
        LDY    3,X
        STY    5,X
        ROL    A

```

```

        STA    3,X
        LDA    #10H
        STA    N      ; Loop counter.

loop:   ROL    4,X
        ROL    5,X      ; Here MSB gets thrown away before you even get started!
        SEC
        LDA    4,X
        SBC    0,X
        TAY
        LDA    5,X
        SBC    1,X
        BCC    skip
        STY    4,X
        STA    5,X
skip:   ROL    2,X
        ROL    3,X

        DEC    N
        BNE    loop

        JMP    POP      ; Eliminate one cell from data stack in ZP and end.
;-----

```

Corrected Forth UM/MOD primitive. What's different from the subroutine version above is that:

1. I/O is on the Forth data stack in ZP, indexed by X, not at fixed addresses.
2. Since X is used for stack indexing, the loop counter is in N.
3. Variable "CARRY" above is now N+1.
4. The N+6 intermediate scratchpad byte above is now N+2.
5. Ends with SWAP to trade quotient and remainder positions on stack.
6. Forth header-construction macro calls and the ending are shown.
7. Standard Forth stack-effect notation is in parentheses on first line. Remember N is a scratchpad area of approximately 10 bytes for local use of Forth code definitions. When a code definition (or a primitive, as it is sometimes called) is done executing, the contents of N are irrelevant. N is not used for parameter-passing from one Forth word to another.

```

HEADER "UM/MOD", NOT_IMMEDIATE      ; ( ud u -- rem quot )
CODE                                ; (Make Forth CFA point to PFA for a code definition.)

        SEC
        LDA    2,X      ; Subtract hi cell of dividend by
        SBC    0,X      ; divisor to see if there's an overflow condition.
        LDA    3,X
        SBC    1,X
        BCS    oflo$    ; Branch if /0 or overflow.

        LDA    #11H     ; Loop 17x.
        STA    N        ; Use N for loop counter.
loop:   ROL    4,X      ; Rotate dividend lo cell left one bit.
        ROL    5,X
        DEC    N        ; Decrement loop counter.
        BEQ    end$     ; If we're done, then branch to end.
        ROL    2,X      ; Otherwise rotate dividend hi cell left one bit.
        ROL    3,X
        STZ    N+1
        ROL    N+1      ; Rotate the bit carried out of above into N+1.

        SEC
        LDA    2,X      ; Subtract dividend hi cell minus divisor.
        SBC    0,X

```

```

        STA      N+2      ; Put result temporarily in N+2 (lo byte)
        LDA      3,X
        SBC      1,X
        TAY                      ; and Y (hi byte).
        LDA      N+1      ; Remember now to bring in the bit carried out above.
        SBC      #0
        BCC      loop

        LDA      N+2      ; If that didn't cause a borrow,
        STA      2,X      ; make the result from above to
        STY      3,X      ; be the new dividend hi cell
        BRA      loop      ; and then brach up.  (NMOS 6502 can use BCS here.)

oflo$:  LDA      #FFH      ; If overflow or /0 condition found,
        STA      2,X      ; just put FFFF in both the remainder
        STA      3,X
        STA      4,X      ; and the quotient.
        STA      5,X

end$:   INX                      ; When you're done, show one less cell on data stack,
        INX                      ; (INX INX is exactly what the Forth word DROP does)
        JMP      SWAP          ; and swap the two top cells to put quotient on top.
                                ; (Actually you'll jump to the beginning of SWAP's
                                ; executable code.  Assembler label "SWAP" is at SWAP's
                                ; PFA, not the CFA that ' SWAP would give you in Forth.
;-----

```

Here's my 65816 UM/MOD . Most Forth primitives (ie, words defined in assembly) are both much shorter and much faster on the '816 than on the '02; but here I chose to speed it up a little more by using the 65816's MVN instruction to move the data into N for work, using X for loop indexing, and eliminating the need to jump to POP at the end, which altogether added nearly 20 lines. (This ends with JMP NEXT. NEXT is Forth's inner loop, not part of the divide routine.) Add also the overflow prediction embodied in the corrected '02 versions above, and you end up with a somewhat long routine. Ultimately with the '816, I may go for a large 1/X table and multiplication table to make things much faster.

Begin with accumulator already in 16-bit mode and the processor in native mode.

The macros shown first here are much more readable than their one-line contents.

```

INDEX_16:  MACRO                      ; Put X & Y into 16-bit mode.
            REP      #00010000B
            NOP                      ; NOP necessary for early versions of
            ENDM                      ; '802 and '816 >4MHz.
;-----
INDEX_8:   MACRO                      ; Put X & Y into 8-bit mode.
            SEP      #00010000B
            NOP                      ; NOP necessary for early versions of
            ENDM                      ; '802 and '816 >4MHz.
;-----

          HEADER "UM/MOD", NOT_IMMEDIATE ; ( ud u -- rem quot )
UMsMOD:  CODE                      ; To save a cycle from every stack load or store, move
        TXY                      ; the inputs to N.  Save stack pointer to restore later
        INY                      ; with one less cell.  (At the exit, N+2 below will be
        INY                      ; transferred to the top of the stack, and N, N+1 where
        STY      XSAVE          ; the divisor is, will be dropped in the bit bucket.)
        ; +-----+-----+-----+-----+-----+-----+
        LDY      #< N          ; |  DIVISOR  |  D I V I D E N D  |temp storage|
        LDA      #5            ; |          |  hi cell      lo cell |of carry bit|
        MVN      0,0           ; |  N      N+1 | N+2      N+3 | N+4      N+5 | N+6      N+7 |
        ; +-----+-----+-----+-----+-----+-----+

```

```

SEC          ; Detect overflow or /0 condition. To find out, sub-
LDA          N+2 ; tract divisor from hi cell of dividend; if C flag
SBC          N   ; remains set, divisor was not big enough to avoid
BCS          uoflo ; overflow. This also takes care of any /0 conditions.
              ; If branch not taken, C flag is left clear for 1st ROL.
              ; We will loop 16x; but since we shift the dividend
LDX          #<$11 ; over at the same time as shifting the answer in, the
              ; operation must start AND finish with a shift of the
              ; lo cell of the dividend (which ends up holding the
INDEX_16     ; quotient), so we start with 17 in X. We will use Y
              ; for temporary storage too, so set index reg.s 16-bit.
ushftl: ROL    N+4 ; Move lo cell of dividend left one bit, also shifting
              ; answer in. The 1st rotation brings in a 0, which
DEX          ; later gets pushed off the other end in the last
BEQ          umend ; rotation. Branch to the end if finished.

ROL          N+2 ; Shift hi cell of dividend left one bit, also shifting
LDA          #0  ; next bit in from high bit of lo cell.
ROL          A
STA          N+6 ; Store old hi bit of dividend in N+6.

SEC          ; See if divisor will fit into hi 17 bits of dividend
LDA          N+2 ; by subtracting and then looking at carry flag.
SBC          N   ; If carry got cleared, divisor did not fit.
TAY          ; Save the difference in Y until we know if we need it.

LDA          N+6 ; Bit 0 of N+6 serves as 17th bit.
SBC          #0  ; Complete the subtraction by doing the 17th bit before
BCC          ushftl ; determining if the divisor fit into the hi 17 bits of
              ; the dividend. If so, the C flag remains set.
STY          N+2 ; If divisor fit into hi 17 bits, update dividend hi
BRA          ushftl ; cell to what it would be after subtraction. Branch.

uoflo: LDA     #$FFFF ; If an overflow or /0 condition occurs, put FFFF in
STA          N+4 ; both the quotient
STA          N+2 ; and the remainder.

umend: INDEX_8
LDX          XSAVE
LDA          N+2 ; Put quotient and remainder on stack where dividend
STA          2,X ; used to be, and restore the stack pointer. Remember
LDA          N+4 ; that we incremented the stack pointer above with TXY,
STA          0,X ; INY, INY, STY XSAVE to knock one cell off the stack.
JMP          NEXT ; (Doing TXY first was to keep X's value for the MVN.)
;-----

```

Last page update: October 18, 2002.